

به نام خدا

تمرین سوم درس پردازش تکاملی

وحید ملکی - 40313004

سوال اول

سؤال ۱ الف) نرخ بازتولید (Reproduction Rate) چیست و زیاد یا کم بودن آن چه مفهومی دارد؟ نرخ بازتولید نشان می‌دهد یک فرد با سطح شایستگی مشخص، پس از انتخاب چند بار در جمعیت تکرار می‌شود. اگر زیاد باشد (بیشتر از ۱)، یعنی فرد شایسته‌تر است و احتمال انتخابش بالاتر است. اگر کم باشد (کمتر از ۱)، یعنی احتمال حذف آن فرد بیشتر است. هدف الگوریتم‌های انتخاب، افزایش نرخ بازتولید افراد قوی و کاهش آن برای افراد ضعیف است.

ب) از دست دادن تنوع (Loss of Diversity) به چه عواملی بستگی دارد؟ از دست دادن تنوع یعنی چند نفر از جمعیت اولیه حذف شوند و تکرار زیادی از تعداد کمی فرد باقی بماند. این موضوع به موارد زیر بستگی دارد:

- روش انتخاب: مثلاً انتخاب تورنمنتی با اندازه بزرگ یا انتخاب با برش (truncation) تنوع را بیشتر کاهش می‌دهند.
- پارامترهای انتخاب: مثلاً مقدار آستانه در روش برش یا اندازه تورنمنت.
- روش‌هایی مثل تقسیم‌بندی جمعیت یا اشتراک‌گذاری شایستگی می‌توانند تنوع را حفظ کنند.

ج) شدت انتخاب (Selection Intensity) چیست و به چه عواملی بستگی دارد؟ شدت انتخاب نشان می‌دهد که انتخاب چقدر باعث افزایش میانگین شایستگی جمعیت شده است. فرمول آن: $I = \frac{\bar{M}^* - \bar{M}}{\sigma}$ که در آن $\bar{M}^*$ میانگین شایستگی بعد از انتخاب، $\bar{M}$ میانگین قبل از انتخاب، و σ انحراف معیار است. عوامل مؤثر:

- توزیع شایستگی اولیه جمعیت
- نوع روش انتخاب (تورنمنتی، نسبی، رتبه‌ای و ...)
- پارامترهای مربوط به هر روش (مثل اندازه تورنمنت یا شیب انتخاب در رتبه‌ای)

سوال دوم

الف) اهمیت حفظ تنوع ژنتیکی در مسائل چندقله‌ای

تنوع ژنتیکی برای جلوگیری از گیر افتادن الگوریتم در بهینه‌های محلی در مسائل چندقله‌ای حیاتی است. کاهش تنوع باعث تمرکز جمعیت در یک ناحیه (بهینه محلی) شده و کاوش نواحی دیگر را محدود می‌کند. تنوع بالا اکتشاف (Exploration) را تقویت کرده و شانس یافتن بهینه‌های جهانی را افزایش می‌دهد. بهره‌برداری (Exploitation) بر بهبود راه‌حل‌های موجود تمرکز دارد، اما بدون تنوع، الگوریتم به سرعت همگرا شده و از کاوش بازمی‌ماند. تعادل بین اکتشاف و بهره‌برداری برای موفقیت در مسائل چندقله‌ای ضروری است.

1. به اشتراک‌گذاری شایستگی (Fitness Sharing)

- **منطق:** شایستگی افراد در نواحی شلوغ با تقسیم بر تعداد افراد نزدیک کاهش می‌یابد تا تنوع حفظ شود.
- **شرایط مؤثر:** وقتی نقاط بهینه نزدیک به هم هستند و نیاز به شناسایی چندین قله است.
- **پارامترها:** شعاع اشتراک (σ_{share}) و تابع اشتراک (مثلاً $sh(d) = 1 - d/\sigma_{share}$ برای $d \leq \sigma_{share}$).
- **مقرون‌به‌صرفه بودن:** برای جمعیت‌های بزرگ محاسبات سنگین است ($O(N^2)$).
- **جلوگیری از تمرکز:** کاهش شایستگی در نواحی شلوغ، افراد در نواحی خلوت را برای انتخاب ترجیح می‌دهد.
- **نوع:** صریح (Explicit).
- **نقاط قوت:** شناسایی دقیق چندین قله.
- **نقاط ضعف:** نیاز به تنظیم دقیق σ_{share} و هزینه محاسباتی بالا.

2. شلوغی (Crowding)

- **منطق:** افراد جدید با افراد مشابه (معمولاً والدین) رقابت می‌کنند تا تنوع حفظ شود.
- **شرایط مؤثر:** وقتی نقاط بهینه پراکنده‌اند و فرزندان شبیه والدین هستند.
- **پارامترها:** معیار فاصله (مثلاً اقلیدسی) و اندازه رقابت (تعداد افراد در هر تورنومنت).
- **مقرون‌به‌صرفه بودن:** محاسبات سبک‌تر ($O(N)$)، مناسب برای جمعیت‌های بزرگ.
- **جلوگیری از تمرکز:** رقابت محلی مانع تسلط یک ناحیه می‌شود.
- **نوع:** صریح (Explicit).
- **نقاط قوت:** ساده و سریع.
- **نقاط ضعف:** وابسته به شباهت فرزندان و والدین، ممکن است تنوع کمتری ایجاد کند.

مقایسه: Fitness Sharing دقیق‌تر اما محاسبات سنگین دارد؛ Crowding سریع‌تر و ساده‌تر است اما ممکن است تنوع کمتری حفظ کند. هر دو به معیار فاصله وابسته‌اند.

پ) طراحی الگوریتم تکاملی

- **روش انتخاب:** از تورنومنت با اندازه کوچک ($k=2$ یا 3) استفاده می‌کنم تا فشار انتخابی کم و تنوع حفظ شود. افزایش k فشار انتخابی را بالا برده و خطر همگرایی زودهنگام را افزایش می‌دهد.
- **مکانیزم حفظ تنوع: مدل جزیره‌ای (Island Model)** را انتخاب می‌کنم، زیرا زیرجمعیت‌ها به صورت موازی نواحی مختلف را کاوش می‌کنند و مهاجرت گاه‌به‌گاه تنوع را حفظ می‌کند.
 - **پارامترها:** تعداد جزایر (5-10)، توپولوژی (حلقه یا شبکه)، نرخ مهاجرت (2-5 فرد)، و فرکانس مهاجرت (هر 25-50 نسل). این مقادیر با آزمایش و بر اساس توزیع نقاط بهینه تنظیم می‌شوند.
- **ساختار چندجمعیتی:** جمعیت به زیرجمعیت‌های برابر (هر کدام 50-100 فرد) تقسیم می‌شود. مهاجرت هر 25-50 نسل با نرخ 5٪ و با انتخاب تصادفی افراد (به‌جای بهترین‌ها) انجام می‌شود تا تنوع حفظ شود. توپولوژی حلقه‌ای برای محدود کردن جریان اطلاعات و حفظ استقلال زیرجمعیت‌ها استفاده می‌شود.

تحلیل: این طراحی با استفاده از تورنومنت با k کوچک و مدل جزیره‌ای، تعادل بین اکتشاف و بهره‌برداری را برقرار کرده و با مهاجرت کنترل‌شده، از همگرایی زودهنگام جلوگیری می‌کند. مدل جزیره‌ای به دلیل موازی‌سازی و حفظ تنوع در بلندمدت، برای مسائل چندقله‌ای بسیار مؤثر است.

سوال سوم

گزارش پیاده‌سازی الگوریتم ژنتیک برای حل مسئله سودوکو

هدف: حل مسئله سودوکو 9×9 با استفاده از الگوریتم ژنتیک (GA) به عنوان یک روش فراابتکاری برای یافتن جدول معتبر با شایستگی نزدیک یا برابر با ۱۶۲ (حداکثر اعتبار).

روش پیاده‌سازی:

- **کروموزوم:** هر کروموزوم یک جدول 9×9 است که خانه‌های ثابت حفظ شده و خانه‌های خالی با اعداد ۱ تا ۹ بدون تکرار در هر سطر پر می‌شوند.
- **تابع شایستگی:** مجموع تعداد اعداد یکتا در ستون‌ها و زیرماتریس‌های 3×3 (حداکثر ۱۶۲).
- **جمعیت اولیه:** ۱۰۰۰ جدول تصادفی با حفظ خانه‌های ثابت و پر کردن تصادفی خانه‌های خالی.
- **انتخاب:** استفاده از روش Stochastic Universal Sampling (SUS) برای انتخاب والدین.
- **تقاطع:** تقاطع یکنواخت برای ترکیب کروموزوم‌ها.
- **جهش:** جابجایی تصادفی دو عدد در سطرها با نرخ ۰.۲.
- **نخبگان:** حفظ ۱۰۰ فرد برتر هر نسل.
- **توقف:** توقف پس از ۱۰۰۰ نسل، شایستگی ۱۶۲، یا ۱۰۰ نسل بدون بهبود.

نتایج:

- **ورودی اولیه:** جدول سودوکو با ۱۰ خانه ثابت.
- **بهترین شایستگی:** ۱۵۸ (نزدیک به راه‌حل کامل).
- **پیشرفت شایستگی:** از ۱۲۴ در نسل صفر به ۱۵۸ در نسل ۲۰۰، با توقف در نسل ۲۵۰ به دلیل عدم بهبود (استگنیشن).
- **جدول نهایی:** جدول تولیدشده تقریباً معتبر است، اما به دلیل شایستگی ۱۵۸، چند تکرار در ستون‌ها یا زیرماتریس‌ها وجود دارد.

نمودار پیشرفت: نمودار شایستگی نشان‌دهنده افزایش پایدار تا نسل ۱۵۰ و سپس ثابت ماندن در ۱۵۸ است.

ارزیابی: الگوریتم به جواب نزدیک به بهینه رسید، اما به دلیل پیچیدگی مسئله و محدودیت‌های جستجو، به شایستگی ۱۶۲ نرسید. استفاده از نرخ جهش بالاتر یا استراتژی‌های ترکیبی (مثل جستجوی محلی) می‌تواند به بهبود نتایج کمک کند.

توضیح مختصر توابع پیاده‌سازی شده برای حل سودوکو با الگوریتم ژنتیک:

1. `is_valid(grid, row, col, num)`

- بررسی می‌کند آیا قرار دادن عدد `num` در خانه `(row, col)` معتبر است یا خیر.
- چک می‌کند: عدم تکرار در سطر، ستون و زیرماتریس 3×3 .
- خروجی: `True` اگر معتبر باشد، `False` در غیر این صورت.

2. `generate_initial_sudoku(filled_cells=10)`

- یک جدول سودوکو اولیه 9×9 با `filled_cells` خانه پرشده تولید می‌کند.
 - خانه‌ها به صورت تصادفی با اعداد ۱ تا ۹ پر می‌شوند، با رعایت قوانین سودوکو.
 - خروجی: ماتریس 9×9 با تعداد محدودی خانه ثابت.
3. `create_individual(initial_sudoku)`
- یک کروموزوم (جدول سودوکو) تولید می‌کند.
 - خانه‌های ثابت را حفظ کرده و خانه‌های خالی هر سطر را با اعداد غیرتکراری ۱ تا ۹ پر می‌کند.
 - خروجی: یک جدول 9×9 به عنوان فرد.
4. `initialize_population(initial_sudoku, population_size)`
- جمعیت اولیه‌ای شامل `population_size` کروموزوم تولید می‌کند.
 - هر کروموزوم با تابع `create_individual` ساخته می‌شود.
 - خروجی: لیست کروموزوم‌ها.
5. `fitness(individual)`
- شایستگی کروموزوم را محاسبه می‌کند.
 - مجموع تعداد اعداد یکتا در ستون‌ها و زیرماتریس‌های 3×3 (حداکثر ۱۶۲).
 - خروجی: عدد صحیح نشان‌دهنده شایستگی.
6. `sus_selection(population, fitnesses, num_parents)`
- والدین را با روش Stochastic Universal Sampling انتخاب می‌کند.
 - بر اساس رتبه‌بندی شایستگی، با احتمال متناسب با رتبه انتخاب می‌کند.
 - خروجی: لیست والدین انتخاب‌شده.
7. `crossover(parent1, parent2)`
- تقاطع یکنواخت بین دو والد انجام می‌دهد.
 - هر سطر از یکی از والدین به صورت تصادفی انتخاب می‌شود.
 - خروجی: یک کروموزوم فرزند.
8. `mutate(individual, initial_sudoku, mutation_rate=0.1)`
- با احتمال `mutation_rate`، دو خانه غیرثابت در یک سطر را جابجا می‌کند.
 - خانه‌های ثابت تغییر نمی‌کنند.
 - خروجی: کروموزوم جهش‌یافته.
9. `genetic_algorithm(initial_sudoku, population_size=1000, max_generations=1000, elitism_size=100, mutation_rate=0.2, stagnation_limit=100)`
- الگوریتم ژنتیک اصلی را اجرا می‌کند.
 - شامل تولید جمعیت، ارزیابی شایستگی، انتخاب، تقاطع، جهش و حفظ نخبگان.
 - تا رسیدن به شایستگی ۱۶۲، حداکثر نسل‌ها یا حد استگنیشن ادامه می‌یابد.
 - خروجی: بهترین کروموزوم و شایستگی آن، همراه با نمایش نمودار پیشرفت شایستگی.

گزارش مختصر و مفید پیاده‌سازی الگوریتم ژنتیک برای مسئله فروشنده دوره‌گرد (TSP)

مقدمه

هدف این گزارش، تحلیل عملکرد الگوریتم ژنتیک طراحی‌شده برای حل مسئله فروشنده دوره‌گرد (TSP) با داده‌های فایل `data.txt` شامل 30 شهر است. این الگوریتم با روش‌های انتخاب تورنمنتی، تقاطع ترتیبی (Order Crossover)، جهش معکوس (Inversion Mutation) و انتخاب بازماندگان ($(\mu + \lambda)$) پیاده‌سازی شده است.

ساختار الگوریتم

- **جمعیت اولیه:** 500 فرد به صورت جایگشت تصادفی از 1 تا 30 تولید شدند.
- **تابع شایستگی:** مجموع هزینه‌های سفر بین شهرهای متوالی در جایگشت به علاوه هزینه بازگشت به شهر اول (هدف: کمینه‌سازی).
- **انتخاب والدین:** روش تورنمنتی با اندازه 3.
- **تقاطع:** عملگر Order Crossover با احتمال 0.8، که یک بازه تصادفی از والد اول را حفظ کرده و بقیه را از والد دوم پر می‌کند.
- **جهش:** جهش معکوس با احتمال 0.2، که یک زیربازه تصادفی را معکوس می‌کند.
- **انتخاب بازماندگان:** روش $(\mu + \lambda)$ برای انتخاب 500 فرد برتر از ترکیب والدین و 1000 فرزند.

نتایج

پس از 100 نسل، بهترین مسیر با هزینه 8945 واحد به دست آمد:

[22, 7, 2, 11, 27, 12, 28, 1, 23, 14, 15, 20, 24, 25, 9, 13, 26, 17, 16, 5, 6, 10, 8, 29, 4, 18, 19, 21, 3, 30]

نمودار تغییرات شایستگی نشان‌دهنده کاهش پیوسته هزینه از نسل‌های اولیه تا نسل 100 است که بیانگر همگرایی مناسب الگوریتم است.

تحلیل عملکرد

- **موفقیت:** الگوریتم مسیری با هزینه 8945 پیدا کرد که نتیجه قابل قبولی است، اما به دلیل ندانستن پاسخ بهینه، نمی‌توان بهینگی آن را تأیید کرد.
- **همگرایی:** هزینه از 9075 (نسل 50) به 8945 (نسل 100) کاهش یافت، اما توقف در بهبود نشان‌دهنده احتمال گیر کردن در بهینه محلی است.
- **محدودیت‌ها:** فضای جستجوی بزرگ مسئله TSP (30 شهر) احتمال گیر کردن در بهینه محلی را افزایش می‌دهد. افزایش نسل‌ها یا تنظیم احتمال جهش می‌تواند نتایج را بهبود بخشد.

توابع و توضیحات

`initialize_population(mu, n)`

1.

• پارامترها:

- `mu`: اندازه جمعیت (تعداد افراد).
- `n`: تعداد شهرها.
- **کارکرد:** تولید جمعیت اولیه با `mu` فرد، که هر فرد یک جایگشت تصادفی از اعداد 1 تا `n` است.
- **خروجی:** لیستی از جایگشت‌ها (افراد).

`fitness(individual, matrix)`

2.

- پارامترها:

- `individual`: یک جایگشت (مسیر) به صورت لیست.
- `matrix`: ماتریس هزینه $(n \times n)$.
- کارکرد: محاسبه مجموع هزینه‌های سفر بین شهرهای متوالی در جایگشت، به علاوه هزینه بازگشت به شهر اول.
- خروجی: مقدار شایستگی (هزینه کل مسیر).

`tournament_selection(population, tournament_size, matrix)`

3.

- پارامترها:

- `population`: لیست افراد جمعیت.
- `tournament_size`: اندازه تورنمنت (تعداد افراد انتخاب شده برای رقابت).
- `matrix`: ماتریس هزینه.
- کارکرد: انتخاب بهترین فرد از میان `tournament_size` فرد تصادفی با مقایسه شایستگی آن‌ها.
- خروجی: بهترین فرد از تورنمنت.

`order_crossover(parent1, parent2)`

4.

- پارامترها:

- `parent1`, `parent2`: دو جایگشت (والدین).
- کارکرد: تولید فرزند با روش تقاطع ترتیبی (Order Crossover); یک بازه تصادفی از `parent1` حفظ شده و بقیه از `parent2` پر می‌شود، به گونه‌ای که جایگشت معتبر بماند.
- خروجی: جایگشت فرزند.

`inversion_mutation(individual)`

5.

- پارامترها:

- `individual`: یک جایگشت.
- کارکرد: معکوس کردن یک زیربازه تصادفی از جایگشت برای ایجاد جهش.
- خروجی: جایگشت جهش یافته.

`genetic_algorithm(matrix, n, mu=100, lambda=100, generations=500, cx_prob=0.8, mt_prob=0.2, stagnation_limit=100)`

6.

- پارامترها:

- `matrix`: ماتریس هزینه.
- `n`: تعداد شهرها.
- `mu`: اندازه جمعیت والدین.
- `lambda`: تعداد فرزندان در هر نسل.
- `generations`: حداکثر تعداد نسل‌ها.
- `cx_prob`: احتمال انجام تقاطع (0.8).
- `mt_prob`: احتمال جهش (0.2).
- `stagnation_limit`: حد توقف در صورت عدم بهبود شایستگی.
- کارکرد: اجرای الگوریتم ژنتیک با مراحل: تولید جمعیت اولیه، انتخاب والدین (تورنمنتی)، تولید فرزندان (تقاطع و جهش)، انتخاب بازماندگان $((\mu + \lambda))$ ، و ثبت بهترین مسیر و هزینه.

- خروجی: بهترین مسیر، هزینه آن، و تاریخچه شایستگی.

ورودی و خروجی

- ورودی: فایل `data.txt` شامل تعداد شهرها $((n))$ ، مقدار مسیر ممنوع، و ماتریس هزینه $(n \times n)$.
- خروجی: بهترین مسیر (جایگشت)، هزینه کل (8945)، و نمودار تغییرات شایستگی با استفاده از `matplotlib`.

نکات

- الگوریتم با 500 والد و 1000 فرزند اجرا شده و پس از 100 نسل به هزینه 8945 می‌رسد.
- نمودار شایستگی نشان‌دهنده همگرایی مناسب است، اما احتمال گیر کردن در بهینه محلی وجود دارد.

سوال پنجم

رمزگشایی کریپتوگرام تک‌حرفی با استفاده از الگوریتم ژنتیک

۱. شرح ساختار کلی الگوریتم ژنتیک طراحی‌شده

الگوریتم ژنتیک طراحی‌شده برای حل مسئله رمزگشایی کریپتوگرام تک‌حرفی شامل مراحل زیر است:

- **تولید جمعیت اولیه:** جمعیت اولیه شامل ۱۰۰۰ فرد (جایگشت‌های تصادفی از حروف A تا Z) است که با استفاده از تابع `random.shuffle` تولید می‌شوند. هر فرد یک جایگشت کامل از ۲۶ حرف الفبای انگلیسی است که به‌عنوان کلید رمزگشایی عمل می‌کند.
- **انتخاب (Selection):** از روش انتخاب تورنمنتی با اندازه تورنمنت ۵ استفاده شده است. در این روش، به‌صورت تصادفی ۵ فرد از جمعیت انتخاب شده و فردی با بهترین (کمترین) مقدار شایستگی به‌عنوان والد انتخاب می‌شود. این روش تعادل خوبی بین تنوع و انتخاب افراد قوی برقرار می‌کند.
- **تقاطع (Crossover):** از تقاطع ترتیبی (Order Crossover) استفاده شده است. در این روش، بخشی از ژن‌های والد اول (یک بازه تصادفی) به فرزند منتقل شده و باقی ژن‌ها از والد دوم به‌ترتیبی که در والد دوم ظاهر می‌شوند، پر می‌شوند، با این شرط که ژن‌های تکراری حذف شوند. این روش از به‌هم‌ریختگی بیش‌ازحد جایگشت جلوگیری می‌کند.
- **جهش (Mutation):** جهش با احتمال ۰.۰۱ انجام می‌شود و شامل جابه‌جایی تصادفی دو حرف در جایگشت است. این کار به حفظ تنوع ژنتیکی و جلوگیری از گیرکردن در مینیمم‌های محلی کمک می‌کند.
- **معیار توقف:** الگوریتم برای ۵۰۰ نسل اجرا می‌شود. همچنین، با استفاده از نخبه‌گرایی (Elitism)، ۵ فرد برتر در هر نسل مستقیماً به نسل بعدی منتقل می‌شوند تا از حفظ بهترین راه‌حل‌ها اطمینان حاصل شود.

۲. توضیح دقیق تابع شایستگی و نحوه محاسبه آن

تابع شایستگی بر اساس مقایسه فراوانی حروف در متن رمزگشایی شده با فراوانی استاندارد حروف در زبان انگلیسی تعریف شده است. فرمول شایستگی به صورت زیر است: $F = \sum_{i=A}^Z (f_i - p_i)^2$ که در آن:

- (f_i) : درصد وقوع حرف (i) در متن رمزگشایی شده.
- (p_i) : درصد وقوع حرف (i) در متن مرجع (کتاب "Pride and Prejudice").

مراحل محاسبه:

1. تولید متن رمزگشایی شده: با استفاده از جایگشت فعلی، هر حرف رمز به حرف متن اصلی نگاشت می شود.
2. شمارش حروف: تعداد وقوع هر حرف (A تا Z) در متن رمزگشایی شده شمرده می شود.
3. محاسبه (f_i) : نسبت تعداد هر حرف به کل تعداد حروف (نه کاراکترها) محاسبه شده و به درصد تبدیل می شود.
4. مقایسه با (p_i) : اختلاف مربع بین (f_i) و (p_i) برای هر حرف محاسبه شده و جمع این مقادیر به عنوان امتیاز شایستگی (F) به دست می آید. هرچه F کمتر باشد، متن رمزگشایی شده به زبان انگلیسی شبیه تر است.

۳. فایل مرجع و روش استخراج فراوانی حروف

فایل مرجع استفاده شده، کتاب "Pride and Prejudice" نوشته جین آستن (pg1342.txt) از پروژه گوتنبرگ است. برای استخراج فراوانی حروف:

1. فایل متنی با استفاده از تابع `process_reference` خوانده شد.
2. تمام کاراکترهای الفبایی (A تا Z، بدون تمایز بین کوچک و بزرگ) شمرده شدند.
3. تعداد کل حروف محاسبه شد و برای هر حرف، درصد وقوع $((p_i))$ به صورت $\text{تعداد} \times \frac{1}{\text{تعداد کل حروف}}$ محاسبه شد.

این فراوانی ها به عنوان معیار استاندارد برای مقایسه با متن رمزگشایی شده استفاده شدند.

۴. بخشی از متن رمزگشایی شده نهایی

بخشی از خروجی فایل `output.txt` به صورت زیر است:

Yofn duone ahr deweh veand aco ofn yatiend bnofcit yonti oh tisd uohtsheht a
hem hatsoh, uohueswer sh Lsbentv, ahr rersuater to tie gnogodstsoh tiat all peh
ane uneater exfal

Hom me ane ehcacer sh a cneat uswsl man, tedtshc mietien tiat hatsoh, on ahv
hatsoh, do uohueswer ahr do rersuater, uah lohc ehrfne. Me ane pet oh a cneat
... battle-yselr oy tiat man. Me iawe uope to rersuate
این متن هنوز کاملاً خوانا نیست، که نشان دهنده نیاز به بهبودهایی در الگوریتم یا تابع شایستگی است.

۵. جدول کلید جایگشت نهایی

جدول نگاشت نهایی از فایل `key.txt` به صورت زیر است:

plaintext:	a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v,
	w, x, y, z
cyphertext:	t, z, q, x, h, l, d, r, k, f, u, j, g, v, s, i, b, c, p, m, e, a,
	y, n, o, w

این جدول نشان می دهد که هر حرف رمز (cyphertext) به چه حرف اصلی (plaintext) نگاشت شده است.

۶. تحلیل عملکرد الگوریتم

- **همگرایی:** الگوریتم از مقدار شایستگی اولیه ۲۴۹.۴۲ در نسل صفر به ۱۱.۶۲ در نسل ۸۲ رسیده است، که نشان دهنده بهبود قابل توجه در شباهت متن رمزگشایی شده به زبان انگلیسی است. نمودار پیشرفت شایستگی (ذخیره شده در `fitness_plot.png`) نشان می دهد که مقدار شایستگی به تدریج کاهش یافته، اما پس از نسل ۸۲، بهبود قابل توجهی مشاهده نشده است.
- **تعداد نسل ها:** الگوریتم برای ۵۰۰ نسل اجرا شد، اما بهترین نتیجه در نسل ۸۲ به دست آمد. این نشان می دهد که تعداد نسل های اضافی تأثیر چندانی نداشته و می توان معیار توقف زودهنگام را اضافه کرد.
- **دقت:** متن رمزگشایی شده هنوز کاملاً خوانا نیست و با متن مورد انتظار (مانند مثال ارائه شده در صورت مسئله) فاصله دارد. این ممکن است به دلیل محدودیت تابع شایستگی تک حرفی باشد که اطلاعات ساختاری مانند دوحرفی ها یا کلمات معنی دار را در نظر نمی گیرد.
- **چالش ها:**
 - **محدودیت تابع شایستگی:** تابع تک حرفی ممکن است برای متون کوتاه یا پیچیده کافی نباشد، زیرا تنها بر فراوانی حروف تمرکز دارد.
 - **یکتایی جایگشت:** همان طور که در صورت مسئله ذکر شد، برخی حروف ممکن است در متن رمز یا اصلی غایب باشند، که باعث می شود چندین جایگشت به نتایج مشابه منجر شوند.
 - **تنظیم پارامترها:** انتخاب مقادیر مناسب برای اندازه جمعیت، نرخ جهش و اندازه تورنمنت نیاز به آزمایش بیشتری دارد.

توضیح مختصر و مفید درباره کد و توابع پیاده سازی شده

کد ارائه شده یک الگوریتم ژنتیک برای رمزگشایی کریپتوگرام تک حرفی پیاده سازی می کند که هدف آن یافتن جایگشتی از حروف الفبا برای تبدیل متن رمز به متن خوانای انگلیسی است. در ادامه، توابع اصلی و نقش آن ها به صورت خلاصه توضیح داده شده است:

1. `process_reference(file_path)`

- **کارکرد:** فایل مرجع (کتاب "Pride and Prejudice") را خوانده و فراوانی استاندارد حروف (A تا Z) را به صورت درصد ((`p_i`)) محاسبه می کند.
- **خروجی:** آرایه ای از درصدهای فراوانی حروف.

2. `process_ciphertext(file_path)`

- **کارکرد:** فایل متن رمز (مثل `ega.txt`) را خوانده و تعداد وقوع هر حرف و کل حروف را شمارش می کند.
- **خروجی:** آرایه شمارش حروف و تعداد کل حروف.

3. `calculate_fitness(key, cipher_counts, total_letters, p)`

- **کارکرد:** شایستگی یک جایگشت (کلید) را با محاسبه اختلاف مربع فراوانی حروف متن رمزگشایی شده ((`f_i`)) و فراوانی مرجع ((`p_i`)) ارزیابی می کند $F = \sum (f_i - p_i)^2$
- **خروجی:** مقدار شایستگی (هرچه کمتر، بهتر).

4. `generate_individual()`

- **کارکرد:** یک جایگشت تصادفی از حروف A تا Z تولید می کند.

- **خروجی:** یک لیست ۲۶ تایی از حروف به صورت جایگشت.
5. `tournament_selection(population, fitness_scores, tournament_size)`
- **کارکرد:** با روش انتخاب تورنمنتی، بهترین فرد از میان (tournament_size) فرد تصادفی را انتخاب می‌کند.
 - **خروجی:** یک جایگشت (والد).
6. `crossover(parent1, parent2)`
- **کارکرد:** با تقاطع ترتیبی، بخشی از ژن‌های والد اول را به فرزند منتقل کرده و باقی ژن‌ها را از والد دوم به ترتیب پر می‌کند.
 - **خروجی:** یک جایگشت جدید (فرزند).
7. `mutate(individual, mutation_rate)`
- **کارکرد:** با احتمال (mutation_rate)، دو حرف تصادفی در جایگشت جابه‌جا می‌شوند.
 - **خروجی:** جایگشت جهش‌یافته.
8. `genetic_algorithm(cipher_counts, total_letters, p, params)`
- **کارکرد:** الگوریتم ژنتیک را با پارامترهای مشخص (جمعیت ۱۰۰۰، نسل ۵۰۰، نرخ جهش ۰.۰۱، تورنمنت ۵، نخبه‌گرایی ۵) اجرا کرده و بهترین جایگشت را پیدا می‌کند. همچنین، نمودار پیشرفت شایستگی را رسم می‌کند.
 - **خروجی:** بهترین جایگشت.
9. `decrypt_file(ciphertext_file, best_individual, output_file)`
- **کارکرد:** با استفاده از بهترین جایگشت، متن رمز را به متن اصلی رمزگشایی کرده و در فایل خروجی ذخیره می‌کند.
 - **خروجی:** فایل متنی رمزگشایی‌شده.
10. `write_key(best_individual, key_file)`
- **کارکرد:** جدول نگاشت جایگشت نهایی (plaintext به cyphertext) را در فایل ذخیره می‌کند.
 - **خروجی:** فایل کلید.
11. `( )main`
- **کارکرد:** مدیریت اجرای برنامه، فراخوانی توابع و تنظیم پارامترها.
 - **خروجی:** فایل‌های output.txt (متن رمزگشایی‌شده) و key.txt (کلید).

جمع‌بندی: کد با استفاده از الگوریتم ژنتیک و تابع شایستگی مبتنی بر فراوانی حروف، متن رمز را رمزگشایی می‌کند. با این حال، خروجی هنوز کاملاً خوانا نیست، که نشان‌دهنده نیاز به تابع شایستگی پیشرفته‌تر (مثل bigram) یا تنظیم بهتر پارامترها است.